

AIE683 Machine Learning

Week 3: Preprocessing, Feature Engineering & Data Leakage

Dr. Hakan Emekci

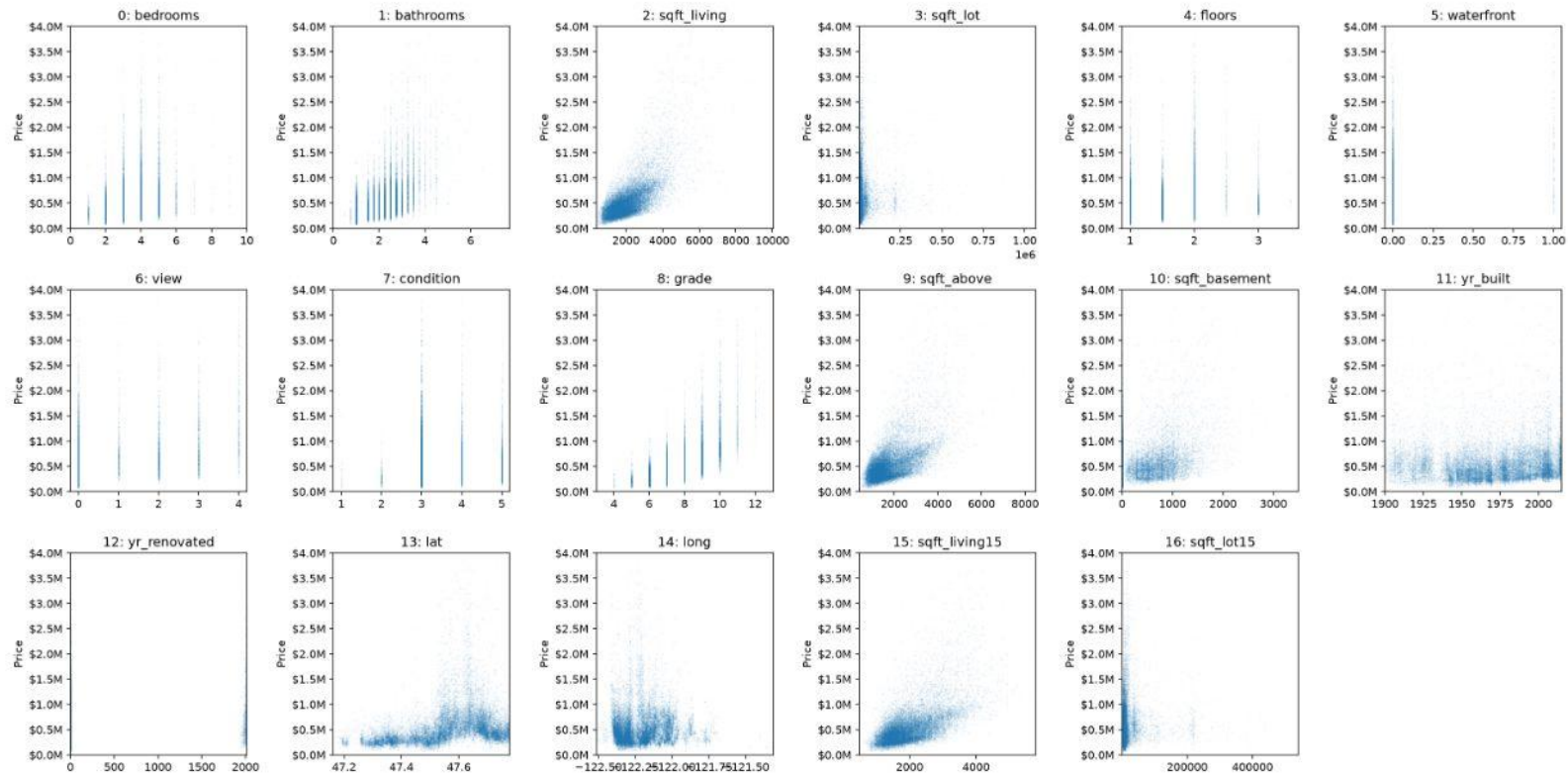
Hacettepe University, Graduate School of Informatics

What is this house worth?

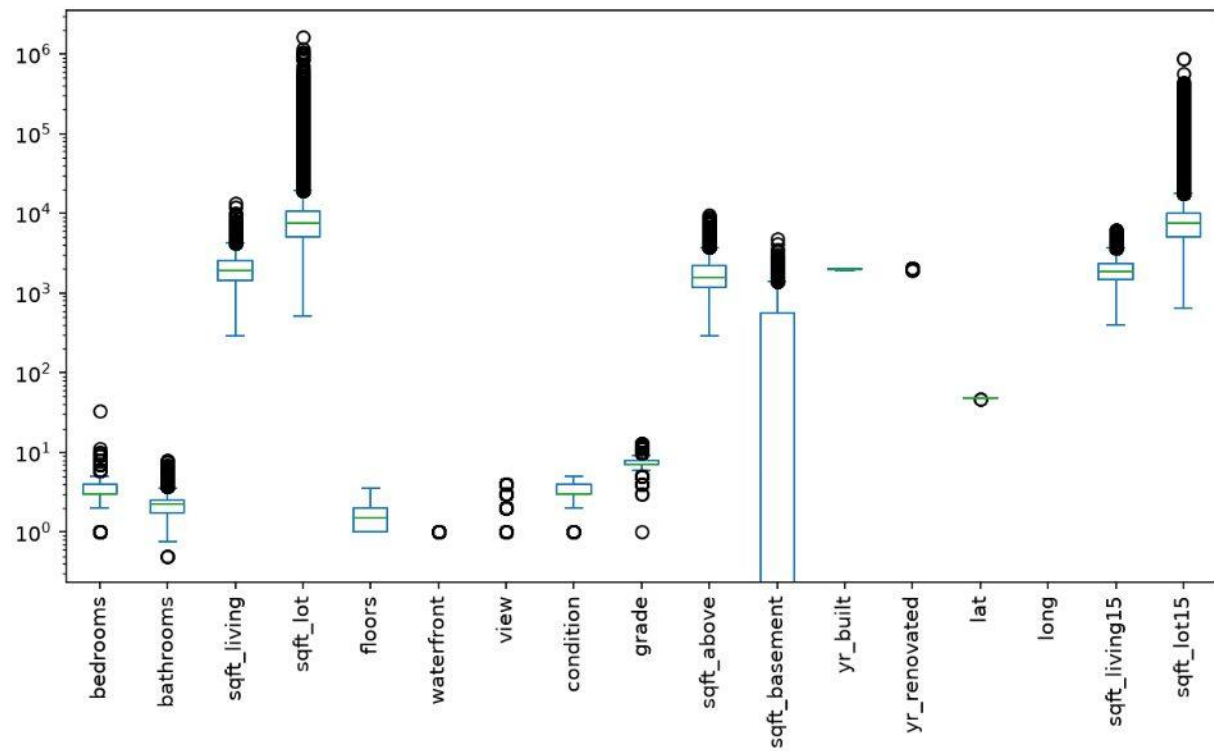
- King County (Seattle area), 21,613 sales between May 2014 and May 2015
- 17 numeric features, a ZIP code, a sale date, and the price
- Real stakes: automated valuation models price homes for lenders, tax offices and buyers, and a few percent of error on a house is a lot of money (we return to what can go wrong in Week 12)

Today's questions:

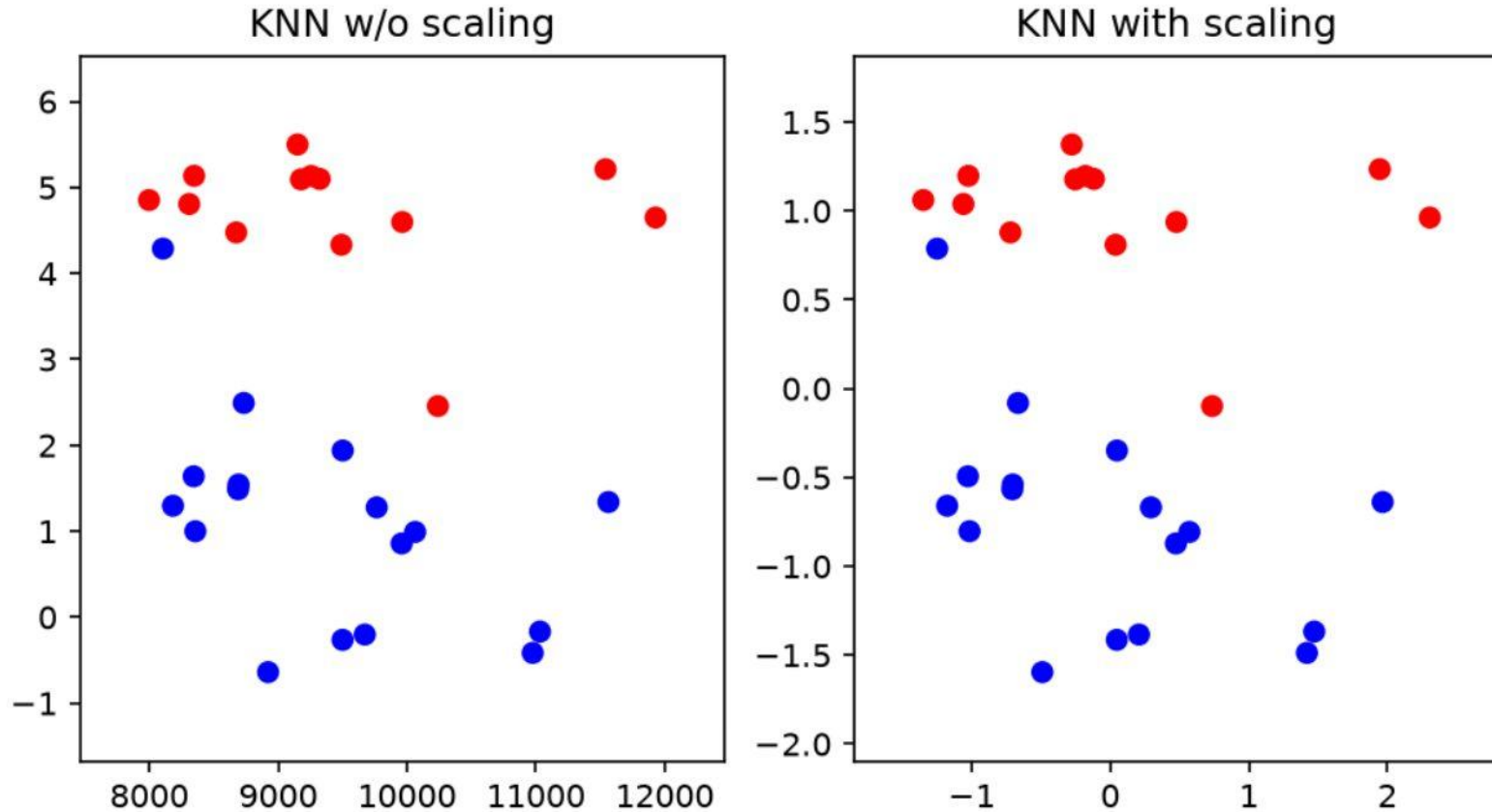
- Which features need rescaling, re-encoding, or transforming?
- How do we do this without fooling ourselves in evaluation?



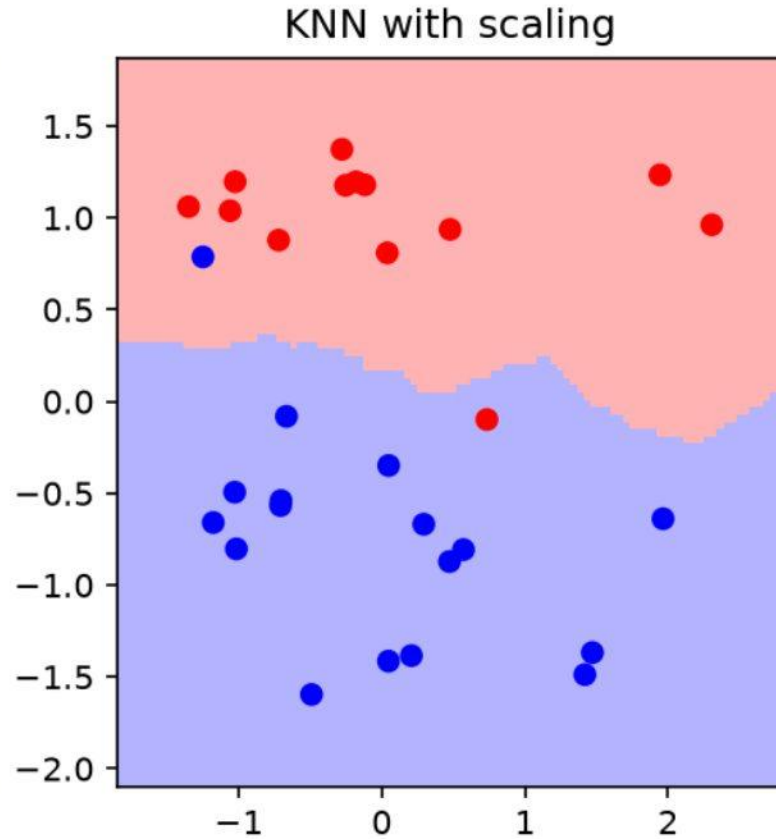
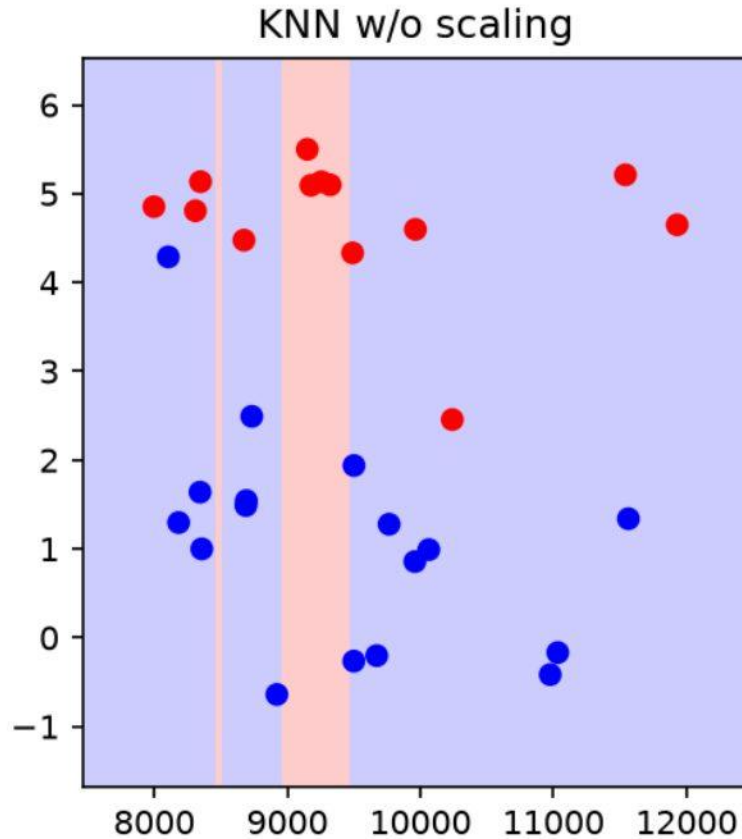
Scaling



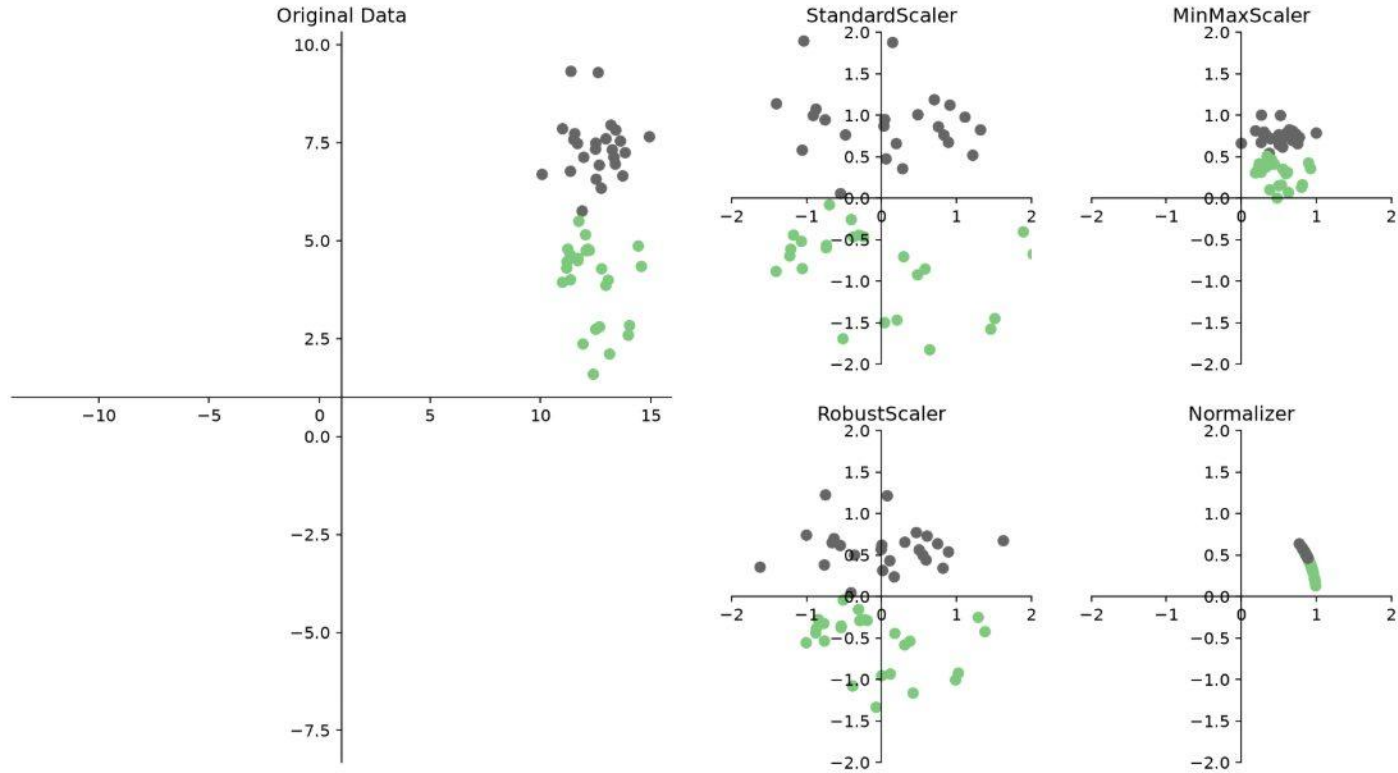
Scaling and Distances



Scaling and Distances



Ways to Scale Data



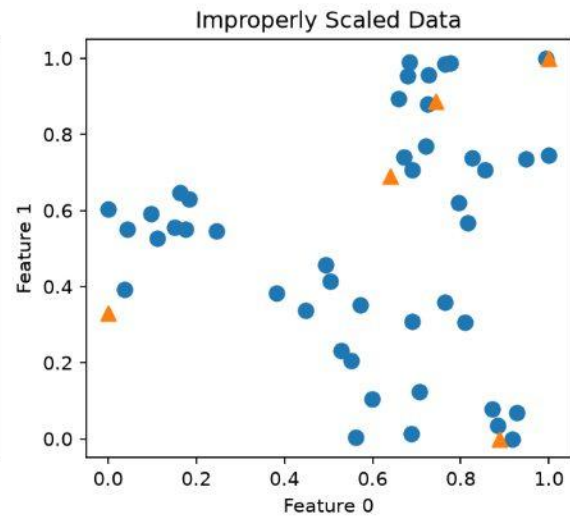
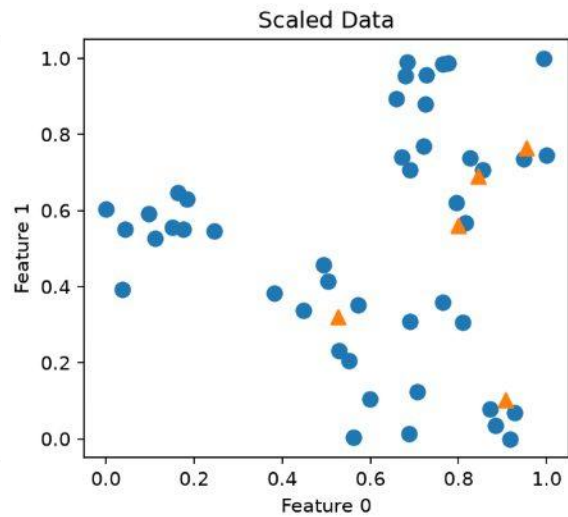
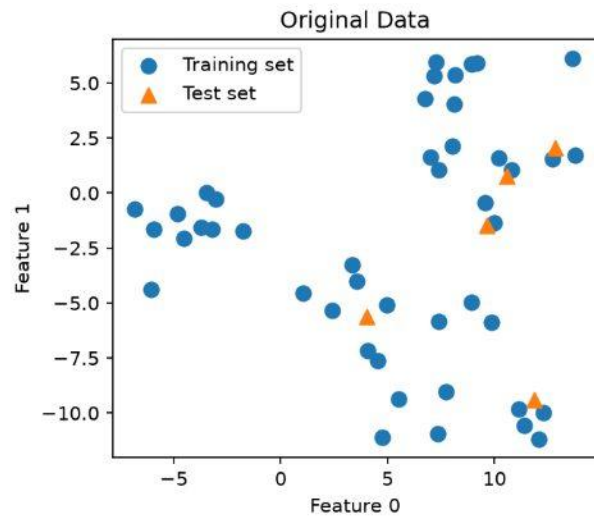
Standard Scaler Example

```
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
X_train, X_test, y_train, y_test = train_test_split(X, y, random_stat

scaler = StandardScaler()
scaler.fit(X_train)
X_train_scaled = scaler.transform(X_train)

ridge = Ridge().fit(X_train_scaled, y_train)
X_test_scaled = scaler.transform(X_test)
ridge.score(X_test_scaled, y_test)
```

0.684



scikit-learn API Summary

`estimator.fit(X, [y])`

`estimator.predict`

`estimator.transform`

Classification

Preprocessing

Regression

Dimensionality reduction

Clustering

Feature selection

Feature extraction

Efficient shortcuts:

```
est.fit_transform(X) == est.fit(X).transform(X) # mostly  
est.fit_predict(X) == est.fit(X).predict(X)    # mostly
```

```
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import RidgeCV
from sklearn.neighbors import KNeighborsRegressor
for name, est, XX in [("RidgeCV, raw", RidgeCV(), X_train),
                      ("RidgeCV, scaled", RidgeCV(), X_train_scaled),
                      ("kNN, raw", KNeighborsRegressor(), X_train),
                      ("kNN, scaled", KNeighborsRegressor(), X_train_scaled)]:
    scores = cross_val_score(est, XX, y_train, cv=10)
    print(f"{name:16s} {scores.mean():.3f} +- {scores.std():.3f}")
```

```
RidgeCV, raw      0.694 +- 0.027
RidgeCV, scaled   0.694 +- 0.027
kNN, raw          0.500 +- 0.039
kNN, scaled       0.786 +- 0.029
```

Try it 1: predict first, then run (10 min)

Three models, each with and without `StandardScaler`:

`KNeighborsRegressor`, `RidgeCV`, `HistGradientBoostingRegressor`.

1. On paper: will scaling help, hurt, or not matter for each?
2. Run the six pipelines in `lab.ipynb` (section "Try it 1").
3. Which model families are invariant to feature scaling, and why?

A note on preprocessing (and pipelines)

A common error

```
rnd = np.random.RandomState(seed=0)
X, y = rnd.normal(size=(100, 10000)), rnd.normal(size=100) # pure noise
X_test, y_test = rnd.normal(size=(100, 10000)), rnd.normal(size=100)
```

```
# select most informative 5% of features
from sklearn.feature_selection import SelectPercentile, f_regression
select = SelectPercentile(score_func=f_regression, percentile=5)
select.fit(X, y)
X_selected = select.transform(X)
print(X_selected.shape)
```

```
(100, 500)
```

```
np.mean(cross_val_score(Ridge(), X_selected, y, cv=5))
```

```
0.91
```

```
ridge = Ridge().fit(X_selected, y)
ridge.score(select.transform(X_test), y_test)
```

```
-0.10
```

Leaking Information

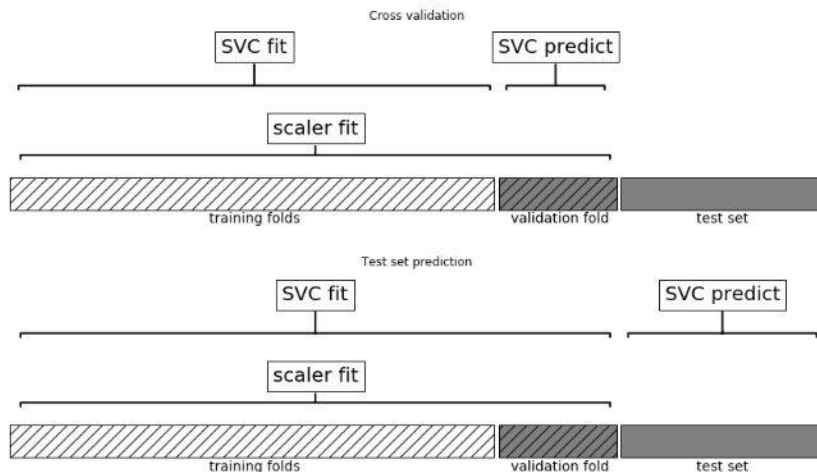
```
# BAD!
select.fit(X, y) # includes the cv test parts!
X_sel = select.transform(X)
scores = []
for train, test in cv.split(X, y):
    ridge = Ridge().fit(X_sel[train], y[train])
    score = ridge.score(X_sel[test], y[test])
    scores.append(score)
```

```
# GOOD!
scores = []
for train, test in cv.split(X, y):
    select.fit(X[train], y[train])
    X_sel_train = select.transform(X[train])
    ridge = Ridge().fit(X_sel_train, y[train])
    X_sel_test = select.transform(X[test])
    score = ridge.score(X_sel_test, y[test])
    scores.append(score)
```

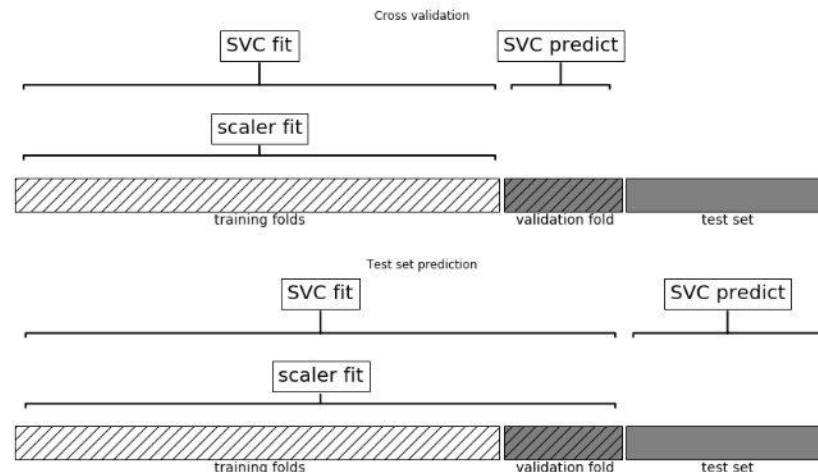
Every step that learns from data belongs inside the cross-validation loop.

Leaking Information

Information leak



No information leak



Cross-validation must simulate the whole training procedure, preprocessing included.

```
scaler = StandardScaler()
scaler.fit(X_train)
X_train_scaled = scaler.transform(X_train)
ridge = Ridge().fit(X_train_scaled, y_train)

X_test_scaled = scaler.transform(X_test)
ridge.score(X_test_scaled, y_test)
```

0.684

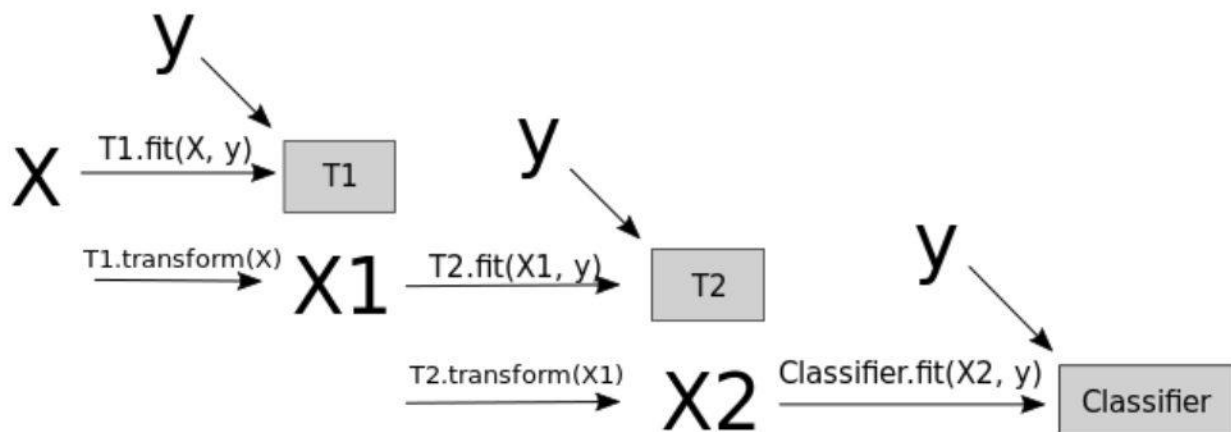
```
from sklearn.pipeline import make_pipeline
pipe = make_pipeline(StandardScaler(), Ridge())
pipe.fit(X_train, y_train)
pipe.score(X_test, y_test)
```

0.684

```
pipe = make_pipeline(T1(), T2(), Classifier())
```



```
pipe.fit(X, y)
```



```
pipe.predict(X')
```



Undoing our feature selection mistake

```
# BAD!
select.fit(X, y) # includes the cv test parts!
X_sel = select.transform(X)
scores = []
for train, test in cv.split(X, y):
    ridge = Ridge().fit(X_sel[train], y[train])
    score = ridge.score(X_sel[test], y[test])
    scores.append(score)
```

Same as:

```
select.fit(X, y)
X_selected = select.transform(X)
np.mean(cross_val_score(Ridge(), X_selected, y, cv=5))
```

0.91

```
# GOOD!
scores = []
for train, test in cv.split(X, y):
    select.fit(X[train], y[train])
    X_sel_train = select.transform(X[train])
    ridge = Ridge().fit(X_sel_train, y[train])
    X_sel_test = select.transform(X[test])
    score = ridge.score(X_sel_test, y[test])
    scores.append(score)
```

Same as:

```
pipe = make_pipeline(select, Ridge())
np.mean(cross_val_score(pipe, X, y, cv=5))
```

-0.25

Naming Steps

```
from sklearn.pipeline import make_pipeline
knn_pipe = make_pipeline(StandardScaler(), KNeighborsRegressor())
print(knn_pipe.steps)
```

```
[('standardscaler', StandardScaler()),
 ('kneighborsregressor', KNeighborsRegressor())]
```

```
from sklearn.pipeline import Pipeline
pipe = Pipeline([("scaler", StandardScaler()),
                  ("regressor", KNeighborsRegressor())])
```

Pipeline and GridSearchCV

```
from sklearn.model_selection import GridSearchCV

knn_pipe = make_pipeline(StandardScaler(), KNeighborsRegressor())
param_grid = {'kneighborsregressor__n_neighbors': range(1, 10)}
grid = GridSearchCV(knn_pipe, param_grid, cv=10)
grid.fit(X_train, y_train)
print(grid.best_params_)
print(grid.score(X_test, y_test))
```

```
{'kneighborsregressor__n_neighbors': 8}
0.778
```

Going wild with Pipelines

```
from sklearn.datasets import load_diabetes
from sklearn.preprocessing import PolynomialFeatures
diabetes = load_diabetes()
X_train, X_test, y_train, y_test = train_test_split(
    diabetes.data, diabetes.target, random_state=0)

pipe = make_pipeline(StandardScaler(), PolynomialFeatures(), Ridge())
param_grid = {'polynomialfeatures__degree': [1, 2, 3],
              'ridge__alpha': [0.001, 0.01, 0.1, 1, 10, 100]}
grid = GridSearchCV(pipe, param_grid=param_grid,
                    n_jobs=-1, return_train_score=True)
grid.fit(X_train, y_train)
```

{'polynomialfeatures__degree': 1, 'ridge__alpha': 10} test R²: 0.358

Going wilder with Pipelines

```
from sklearn.linear_model import Lasso
pipe = Pipeline([('scaler', StandardScaler()),
                  ('regressor', Ridge())])

param_grid = {'scaler': [StandardScaler(), MinMaxScaler(),
                          'passthrough'],
               'regressor': [Ridge(), Lasso()],
               'regressor__alpha': np.logspace(-3, 3, 7)}

grid = GridSearchCV(pipe, param_grid)
grid.fit(X_train, y_train)
grid.score(X_test, y_test)
```

Keeping column names: `set_output`

```
scaler = StandardScaler().set_output(transform="pandas")
scaler.fit(X_train).transform(X_test).iloc[:3, :5]
```

	bedrooms	bathrooms	sqft_living	sqft_lot	floors
17384	-1.448419	-0.801701	-0.704248	-0.321932	2.769831
722	0.667617	1.467506	2.799781	0.864530	0.925552
2680	-1.448419	-1.774219	-0.693433	-0.273468	-0.918726

- Works on transformers, pipelines, and `ColumnTransformer`
- Global switch: `sklearn.set_config(transform_output="pandas")`
- `get_feature_names_out()` gives the names without converting

Categorical Variables

Categorical Variables

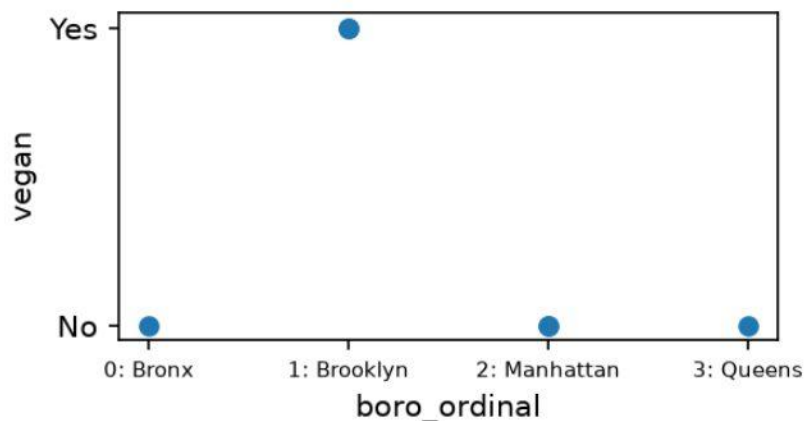
```
import pandas as pd
df = pd.DataFrame({
    'boro': ['Manhattan', 'Queens', 'Manhattan', 'Brooklyn', 'Brooklyn', 'Bronx'],
    'salary': [103, 89, 142, 54, 63, 219],
    'vegan': ['No', 'No', 'No', 'Yes', 'Yes', 'No']})
```

	boro	salary	vegan
0	Manhattan	103	No
1	Queens	89	No
2	Manhattan	142	No
3	Brooklyn	54	Yes
4	Brooklyn	63	Yes
5	Bronx	219	No

Ordinal encoding

```
df['boro_ordinal'] = df.boro.astype("category").cat.codes  
df
```

	boro	salary	vegan	boro_ordinal
0	Manhattan	103	No	2
1	Queens	89	No	3
2	Manhattan	142	No	2
3	Brooklyn	54	Yes	1
4	Brooklyn	63	Yes	1
5	Bronx	219	No	0



One-Hot (Dummy) Encoding

	boro	salary	vegan
0	Manhattan	103	No
1	Queens	89	No
2	Manhattan	142	No
3	Brooklyn	54	Yes
4	Brooklyn	63	Yes
5	Bronx	219	No

```
pd.get_dummies(df, dtype=int)
```

	salary	boro_Bronx	boro_Brooklyn	boro_Manhattan	boro_Queens	vegan_No	vegan_Yes
0	103	0	0	1	0	1	0
1	89	0	0	0	1	1	0
2	142	0	0	1	0	1	0
3	54	0	1	0	0	0	1
4	63	0	1	0	0	0	1
5	219	1	0	0	0	1	0

OneHotEncoder

```
from sklearn.preprocessing import OneHotEncoder
ohe = OneHotEncoder(sparse_output=False).set_output(transform="pandas")
ohe.fit_transform(df[['boro', 'vegan']])
```

	boro_Bronx	boro_Brooklyn	boro_Manhattan	boro_Queens	vegan_No	vegan_Y
0	0.0	0.0	1.0	0.0	1.0	0
1	0.0	0.0	0.0	1.0	1.0	0
2	0.0	0.0	1.0	0.0	1.0	0
3	0.0	1.0	0.0	0.0	0.0	1
4	0.0	1.0	0.0	0.0	0.0	1
5	1.0	0.0	0.0	0.0	1.0	0

- `fit` learns the categories; `transform` always produces the same columns
- Encodes every column it is given: use it on categorical columns only
- Sparse output by default (`sparse_output=True`)

Unknown and rare categories

```
ohe = OneHotEncoder(handle_unknown="infrequent_if_exist", min_frequency=2,  
                    sparse_output=False)
```

```
ohe.fit(df[['boro']])
```

```
ohe.get_feature_names_out(), ohe.infrequent_categories_
```

```
(['boro_Brooklyn', 'boro_Manhattan', 'boro_infrequent_sklearn'],  
 [array(['Bronx', 'Queens'])])
```

```
ohe.transform(pd.DataFrame({'boro': ['Manhattan', 'Staten Island', 'Queens']}))
```

```
array([[0., 1., 0.],  
       [0., 0., 1.],      # unseen -> infrequent column  
       [0., 0., 1.]])
```

- `handle_unknown="error"` (default), `"ignore"` (all zeros), `"infrequent_if_exist"`
- `min_frequency` / `max_categories` cap the number of columns

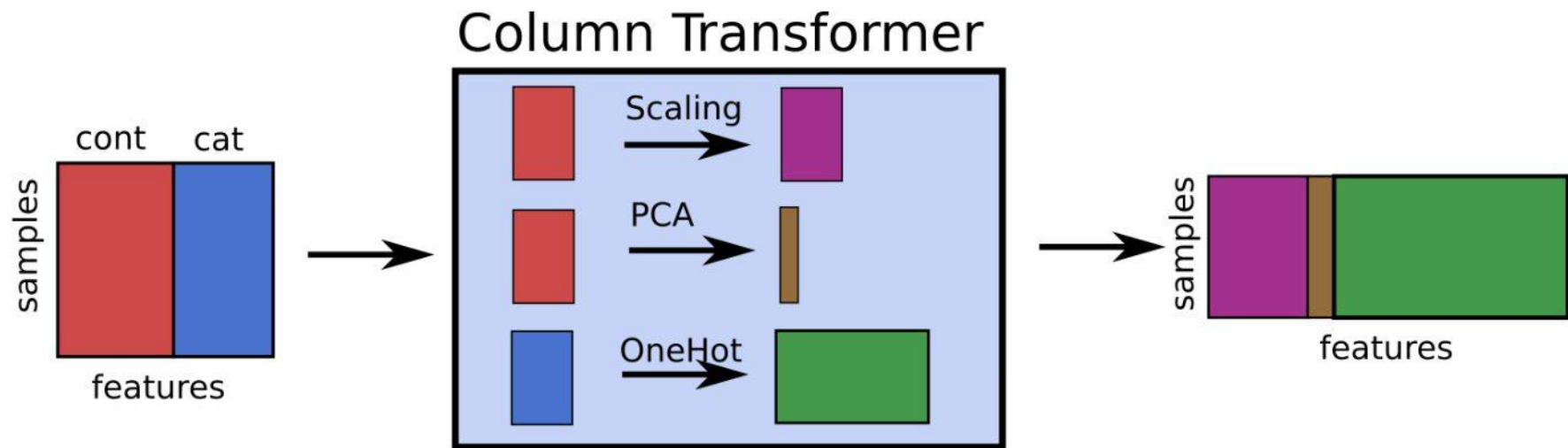
OneHotEncoder + ColumnTransformer

```
from sklearn.compose import make_column_transformer
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression

numeric = df.select_dtypes(include="number").columns
categorical = df.select_dtypes(exclude="number").columns

preprocess = make_column_transformer(
    (StandardScaler(), numeric),
    (OneHotEncoder(handle_unknown="ignore"), categorical))

model = make_pipeline(preprocess, LogisticRegression())
```



ColumnTransformer: full syntax and column selectors

```
from sklearn.compose import ColumnTransformer, make_column_selector

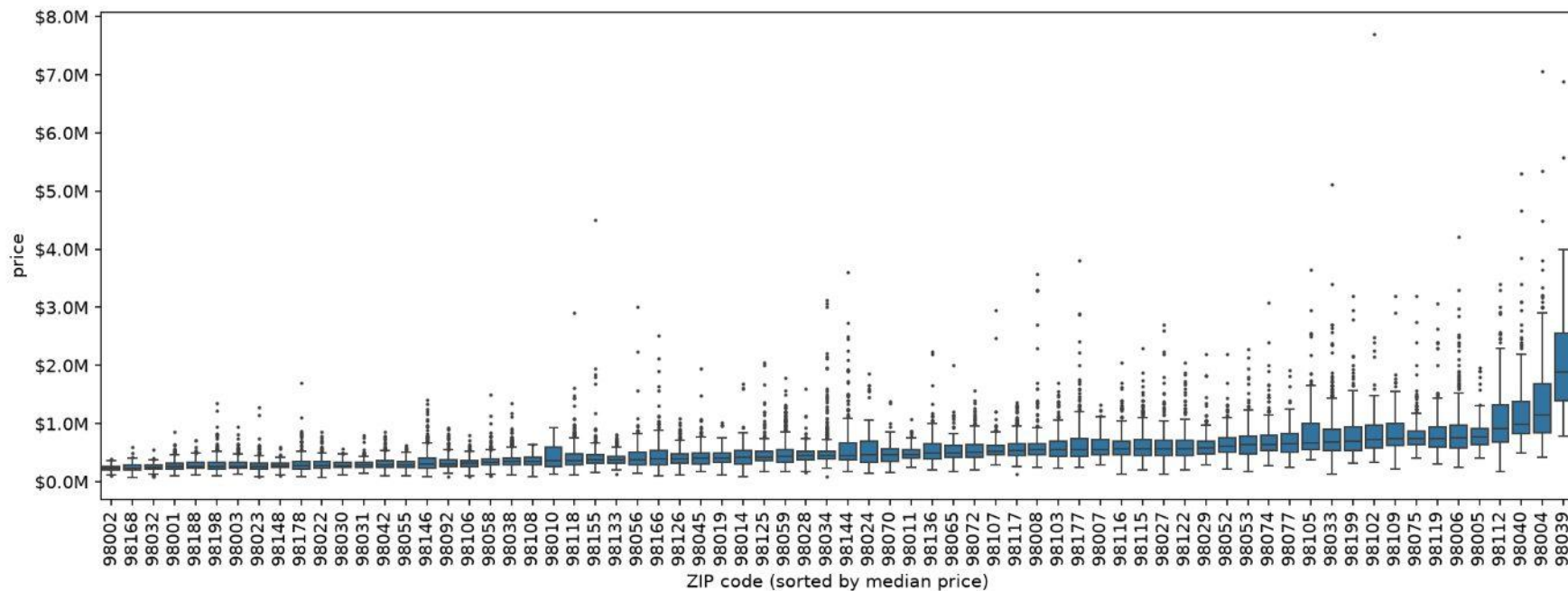
preprocess = ColumnTransformer([
    ("num", StandardScaler(), make_column_selector(dtype_include="number")),
    ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False),
     make_column_selector(dtype_include=object)),
], remainder="drop").set_output(transform="pandas")

model = make_pipeline(preprocess, LogisticRegression())
model.fit(df[['boro', 'salary']], df.vegan)
```

```
param_grid = {"columntransformer__cat__min_frequency": [None, 2],
              "logisticregression__C": [0.1, 1, 10]}
```

- Parameter names: <pipeline step>__<transformer name>__<parameter>
- `make_column_selector` resolves columns at fit time (by dtype or regex `pattern=`)

Target Encoding (Impact Encoding)



Target Encoding (Impact Encoding)

- For high-cardinality categorical features
- Replace each category by a statistic of the target in that category:
 - Regression: mean target ("average price in this ZIP code")
 - Binary classification: fraction of positives
 - Multiclass: one column per class
- Shrink toward the global mean for rare categories
- The encoding uses y : it must be fit on training data only

TargetEncoder and cross-fitting

```
from sklearn.preprocessing import TargetEncoder
te = TargetEncoder(target_type="continuous", cv=KFold(5, shuffle=True, random_state=0))
ct = make_column_transformer((te, ["zipcode"]), remainder="passthrough")
X_train_te = ct.fit_transform(X_train, y_train) # cross-fitted
X_test_te  = ct.transform(X_test)             # full-training-set encoding
```

- `fit_transform(X, y)`: each fold is encoded with statistics from the other folds
- `transform(X)`: uses statistics from the whole training set
- so `fit(X, y).transform(X) ≠ fit_transform(X, y)`
- set `target_type="continuous"` for regression

	zipcode	encoded	train mean
0	98125	470230.0	470154.0
1	98005	803606.0	804744.0
2	98107	573870.0	573923.0
3	98109	869053.0	875433.0
4	98155	424224.0	424018.0

Using the ZIP code

```
cv = KFold(5, shuffle=True, random_state=0)
num_cols = make_column_selector(dtype_include="number")
pipe_ohe = make_pipeline(make_column_transformer(
    (OneHotEncoder(handle_unknown="ignore"), ["zipcode"]),
    (StandardScaler(), num_cols)), Ridge())
pipe_te = make_pipeline(make_column_transformer(
    (TargetEncoder(target_type="continuous", cv=cv), ["zipcode"]),
    (StandardScaler(), num_cols)), Ridge())
```

```
no zipcode: 0.693
one-hot:    0.805
target enc: 0.786
```

- Both encodings add over 0.09 R^2
- With 70 levels and 21,613 rows, one-hot is affordable and slightly better here
- Target encoding wins when levels are many and rows per level few

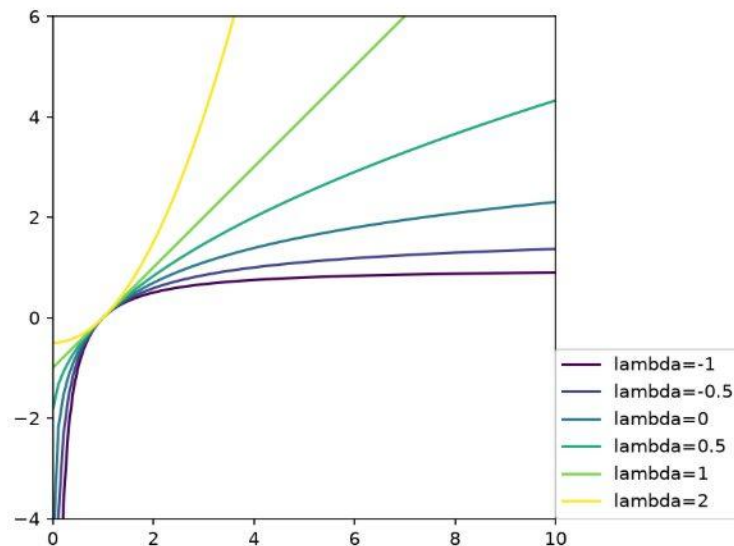
Feature Distributions

Power Transformations

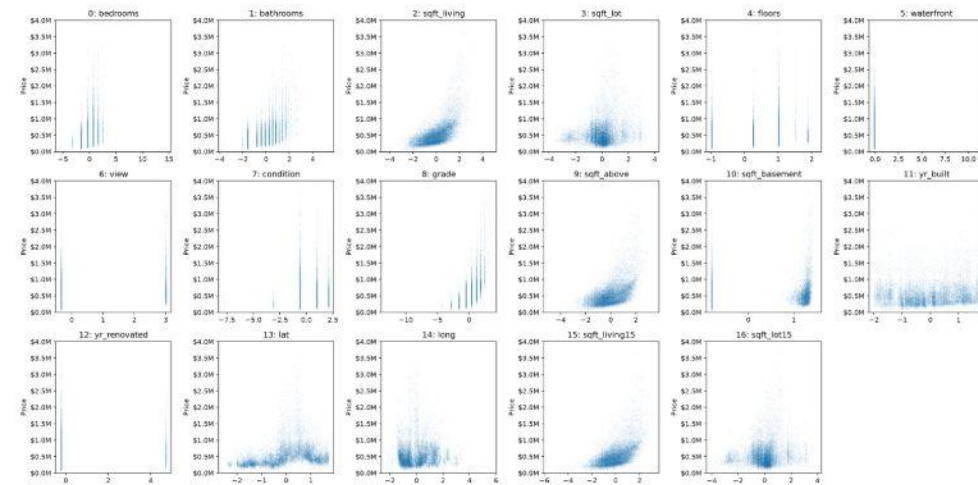
$$\text{bc}_\lambda(x) = \begin{cases} \frac{x^\lambda - 1}{\lambda} & \text{if } \lambda \neq 0 \\ \log(x) & \text{if } \lambda = 0 \end{cases}$$

Box-Cox: only for positive x.

Yeo-Johnson: any x (default).



```
from sklearn.preprocessing import PowerTransformer
pt = PowerTransformer(method="yeo-johnson") # also standardizes by
pt.fit(X_train)
```



Features after the
transform vs price

StandardScaler
PowerTransformer
PowerTransformer + $\log(y)$

$R^2 = 0.6$

$R^2 = 0.6$

$R^2 = 0.7$

Missing Values

Dealing with missing values

- Missing values can be encoded in many ways
- NumPy has no dedicated missing value; the convention is `np.nan`.
pandas has `pd.NA` and nullable dtypes
- Sometimes: 999, ???, ?, `np.inf`, "N/A", "Unknown" ...
- Not discussing "missing output" - that's semi-supervised learning.
- Often missingness is informative (use `MissingIndicator` or `SimpleImputer(add_indicator=True)`)

```

array([[ nan, 3.2, 5.7, 2.3],
       [ nan, 2.8, 4.9, 2. ],
       [ nan, 2.8, 6.7, 2. ],
       [ nan, 2.7, 4.9, 1.8],
       [ 6.7, 3.3, 5.7, 2.1],
       [ nan, 3.2, 6. , 1.8],
       [ nan, 2.8, 4.8, 1.8],
       [ nan, 3. , 4.9, 1.8],
       [ nan, 2.8, 5.6, 2.1],
       [ nan, 3. , 5.8, 1.6],
       [ 7.4, 2.8, 6.1, 1.9],
       [ nan, 3.8, 6.4, 2. ],
       [ 6.4, 2.8, 5.6, 2.2],
       [ nan, 2.8, 5.1, 1.5],
       [ nan, 2.6, 5.6, 1.4],
       [ nan, 3. , 6.1, 2.3],
       [ nan, 3.4, 5.6, 2.4],
       [ nan, 3.1, 5.5, 1.8],
       [ nan, 3. , 4.8, 1.8],
       [ 6.9, 3.1, 5.4, 2.1],
       [ 6.7, 3.1, 5.6, 2.4],
       [ nan, 3.1, 5.1, 2.3],
       [ nan, 2.7, 5.1, 1.9],
       [ nan, 3.2, 5.9, 2.3],
       [ nan, 3.3, 5.7, 2.5],
       [ nan, 3. , 5.2, 2.3],
       [ nan, 2.5, 5. , 1.9],
       [ nan, 3. , 5.2, 2. ],
       [ 6.2, 3.4, 5.4, 2.3],
       [ nan, 3. , 5.1, 1.8]])

```

```

array([[ 5.1, 3.5, 1.4, 0.2],
       [ nan, nan, 1.4, 0.2],
       [ 4.7, 3.2, 1.3, 0.2],
       [ 4.6, 3.1, 1.5, 0.2],
       [ 5. , 3.6, 1.4, 0.2],
       [ nan, nan, nan, nan],
       [ 4.6, 3.4, 1.4, 0.3],
       [ 5. , 3.4, 1.5, 0.2],
       [ 4.4, 2.9, 1.4, 0.2],
       [ 4.9, 3.1, 1.5, 0.1],
       [ 5.4, 3.7, 1.5, 0.2],
       [ 4.8, 3.4, 1.6, 0.2],
       [ 4.8, 3. , 1.4, 0.1],
       [ 4.3, 3. , 1.1, 0.1],
       [ nan, nan, nan, nan],
       [ 5.7, 4.4, 1.5, 0.4],
       [ 5.4, 3.9, 1.3, 0.4],
       [ 5.1, 3.5, 1.4, 0.3],
       [ 5.7, 3.8, 1.7, 0.3],
       [ 5.1, 3.8, 1.5, 0.3],
       [ 5.4, 3.4, 1.7, 0.2],
       [ 5.1, 3.7, 1.5, 0.4],
       [ 4.6, 3.6, 1. , 0.2],
       [ 5.1, nan, nan, nan],
       [ 4.8, 3.4, 1.9, 0.2],
       [ 5. , 3. , 1.6, 0.2],
       [ nan, nan, nan, 0.4],
       [ 5.2, 3.5, 1.5, 0.2],
       [ 5.2, 3.4, 1.4, 0.2],
       [ 4.7, 3.2, 1.6, 0.2]])

```

```
array([[ 6. ,  3.4,  4.5, nan],
       [ 6.9,  3.1,  5.1,  2.3],
       [ 4.6,  3.2,  1.4, nan],
       [ 5.1,  3.8,  1.5, nan],
       [ 4.4,  2.9, nan, nan],
       [ 6.6,  2.9,  4.6,  1.3],
       [ 6.7,  3. ,  5.2,  2.3],
       [ 6.3,  3.3,  6. ,  2.5],
       [ 7.2,  3. ,  5.8,  1.6],
       [ 4.6,  3.4,  1.4, nan],
       [ 5.2,  3.5,  1.5, nan],
       [ 5.4,  3.4, nan, nan],
       [ 5.9,  3.2,  4.8,  1.8],
       [ 4.9,  3.1, nan, nan],
       [ 6.9,  3.2,  5.7,  2.3],
       [ 5.7,  3.8, nan,  0.3],
       [ 5.3,  3.7,  1.5, nan],
       [ 4.5,  2.3,  1.3, nan],
       [ 6.5,  3. ,  5.5,  1.8],
       [ 6.2,  2.9,  4.3,  1.3],
       [ 6.4,  2.8,  5.6,  2.2],
       [ 6.1,  3. ,  4.6,  1.4],
       [ 6.2,  2.8,  4.8,  1.8],
       [ 4.9,  2.5,  4.5,  1.7],
       [ 6. ,  2.7,  5.1, nan],
       [ 6.8,  3.2,  5.9,  2.3],
       [ 6. ,  2.9,  4.5,  1.5],
       [ 4.9,  2.4,  3.3,  1. ],
       [ 5.8,  2.7,  5.1, nan],
       [ 5.5,  2.4,  3.8, nan]])
```



Imputation

```
array([[ 6. ,  3.4,  4.5,  1.6],
       [ 6.9,  3.1,  5.1,  2.3],
       [ 4.6,  3.2,  1.4,  0.2],
       [ 5.1,  3.8,  1.5,  0.3],
       [ 4.4,  2.9,  1.4,  0.2],
       [ 6.6,  2.9,  4.6,  1.3],
       [ 6.7,  3. ,  5.2,  2.3],
       [ 6.3,  3.3,  6. ,  2.5],
       [ 7.2,  3. ,  5.8,  1.6],
       [ 4.6,  3.4,  1.4,  0.3],
       [ 5.2,  3.5,  1.5,  0.2],
       [ 5.4,  3.4,  1.5,  0.4],
       [ 5.9,  3.2,  4.8,  1.8],
       [ 4.9,  3.1,  1.5,  0.1],
       [ 6.9,  3.2,  5.7,  2.3],
       [ 5.7,  3.8,  1.7,  0.3],
       [ 5.3,  3.7,  1.5,  0.2],
       [ 4.5,  2.3,  1.3,  0.3],
       [ 6.5,  3. ,  5.5,  1.8],
       [ 6.2,  2.9,  4.3,  1.3],
       [ 6.4,  2.8,  5.6,  2.2],
       [ 6.1,  3. ,  4.6,  1.4],
       [ 6.2,  2.8,  4.8,  1.8],
       [ 4.9,  2.5,  4.5,  1.7],
       [ 6. ,  2.7,  5.1,  1.6],
       [ 6.8,  3.2,  5.9,  2.3],
       [ 6. ,  2.9,  4.5,  1.5],
       [ 4.9,  2.4,  3.3,  1. ],
       [ 5.8,  2.7,  5.1,  1.9],
       [ 5.5,  2.4,  3.8,  1.1]])
```

Imputation Methods

- Mean / Median (per column)
- kNN
- Regression models, iterated (IterativeImputer)
- Matrix factorization (not in this lecture)

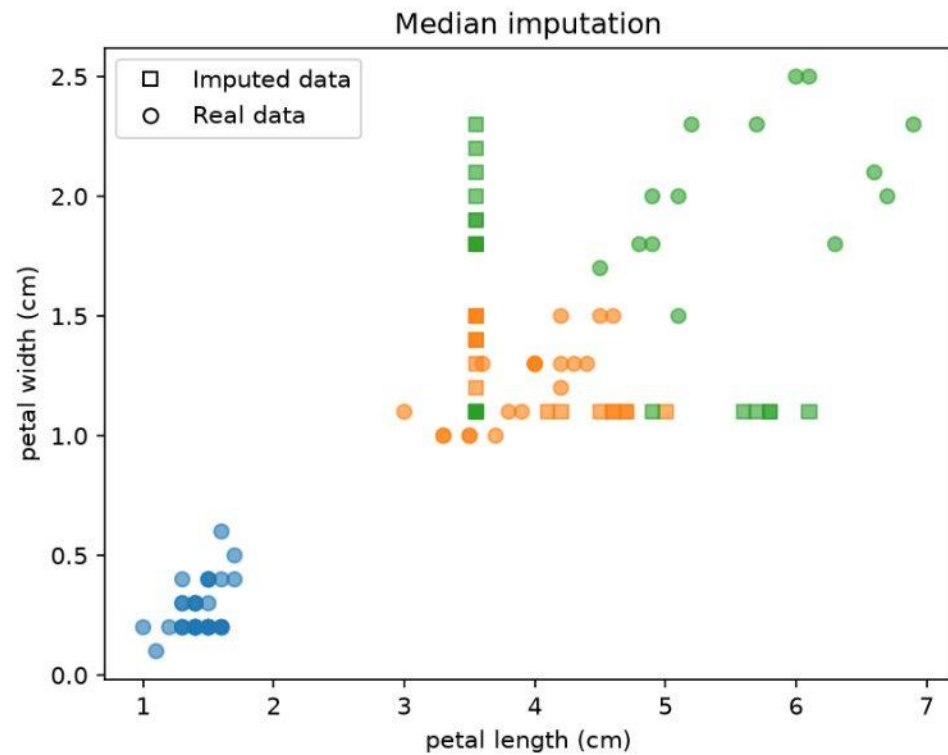
Mean and Median

```
[[ 6.  2.9  4.5  1.5]
 [ 5.9  3.  5.1  1.8]
 [ 4.4  3.  1.3  0.2]
 [ 5.1  3.3  nan  nan]
 [ 5.  3.5  1.6  0.6]
 [ 5.4  3.4  nan  nan]
 [ 5.7  3.8  nan  0.3]
 [ 5.6  2.5  3.9  nan]
 [ 7.7  2.6  6.9  2.3]
 [ 5.8  2.7  5.1  1.9]
 [ 6.7  3.1  5.6  2.4]
 [ 4.8  3.4  1.9  nan]
 [ 7.2  3.2  6.  1.8]
 [ 4.4  2.9  nan  nan]
 [ 6.9  3.2  5.7  2.3]
 [ 5.5  4.2  1.4  nan]
 [ 6.3  2.3  4.4  1.3]
 [ 7.  3.2  4.7  1.4]
 [ 5.8  2.7  nan  nan]
 [ 6.8  2.8  4.8  1.4]
 [ 5.4  3.9  1.7  nan]
 [ 7.6  3.  6.6  2.1]
 [ 7.7  2.8  6.7  2. ]
 [ 5.  3.3  nan  0.2]
 [ 5.9  3.  4.2  1.5]
 [ 6.1  2.8  4.  1.3]
 [ 5.  3.6  1.4  0.2]
 [ 7.4  2.8  6.1  1.9]
 [ 6.3  2.5  5.  1.9]
 [ 6.7  3.3  5.7  2.5]]
```

```
from sklearn.impute import SimpleImputer
imp = SimpleImputer(strategy="median").fit(X_train)
X_median_imp = imp.transform(X_train)
```

Imputation

```
array([[ 6.  ,  2.9 ,  4.5 ,  1.5 ],
 [ 5.9 ,  3.  ,  5.1 ,  1.8 ],
 [ 4.4 ,  3.  ,  1.3 ,  0.2 ],
 [ 5.1 ,  3.3 ,  4.116,  1.462],
 [ 5.  ,  3.5 ,  1.6 ,  0.6 ],
 [ 5.4 ,  3.4 ,  4.116,  1.462],
 [ 5.7 ,  3.8 ,  4.116,  0.3 ],
 [ 5.6 ,  2.5 ,  3.9 ,  1.462],
 [ 7.7 ,  2.6 ,  6.9 ,  2.3 ],
 [ 5.8 ,  2.7 ,  5.1 ,  1.9 ],
 [ 6.7 ,  3.1 ,  5.6 ,  2.4 ],
 [ 4.8 ,  3.4 ,  1.9 ,  1.462],
 [ 7.2 ,  3.2 ,  6.  ,  1.8 ],
 [ 4.4 ,  2.9 ,  4.116,  1.462],
 [ 6.9 ,  3.2 ,  5.7 ,  2.3 ],
 [ 5.5 ,  4.2 ,  1.4 ,  1.462],
 [ 6.3 ,  2.3 ,  4.4 ,  1.3 ],
 [ 7.  ,  3.2 ,  4.7 ,  1.4 ],
 [ 5.8 ,  2.7 ,  4.116,  1.462],
 [ 6.8 ,  2.8 ,  4.8 ,  1.4 ],
 [ 5.4 ,  3.9 ,  1.7 ,  1.462],
 [ 7.6 ,  3.  ,  6.6 ,  2.1 ],
 [ 7.7 ,  2.8 ,  6.7 ,  2. ],
 [ 5.  ,  3.3 ,  4.116,  0.2 ],
 [ 5.9 ,  3.  ,  4.2 ,  1.5 ],
 [ 6.1 ,  2.8 ,  4.  ,  1.3 ],
 [ 5.  ,  3.6 ,  1.4 ,  0.2 ],
 [ 7.4 ,  2.8 ,  6.1 ,  1.9 ],
 [ 6.3 ,  2.5 ,  5.  ,  1.9 ],
 [ 6.7 ,  3.3 ,  5.7 ,  2.5 ]])
```



Comparing strategies in a pipeline

```
from sklearn.impute import SimpleImputer, KNNImputer
nan_columns = np.any(np.isnan(X_train), axis=0)
drop = make_pipeline(StandardScaler(), LogisticRegression())
cross_val_score(drop, X_train[:, ~nan_columns], y_train, cv=10).mean()    # 0.830

median = make_pipeline(SimpleImputer(strategy='median'), StandardScaler(), LogisticRegression())
cross_val_score(median, X_train, y_train, cv=10).mean()                  # 0.830

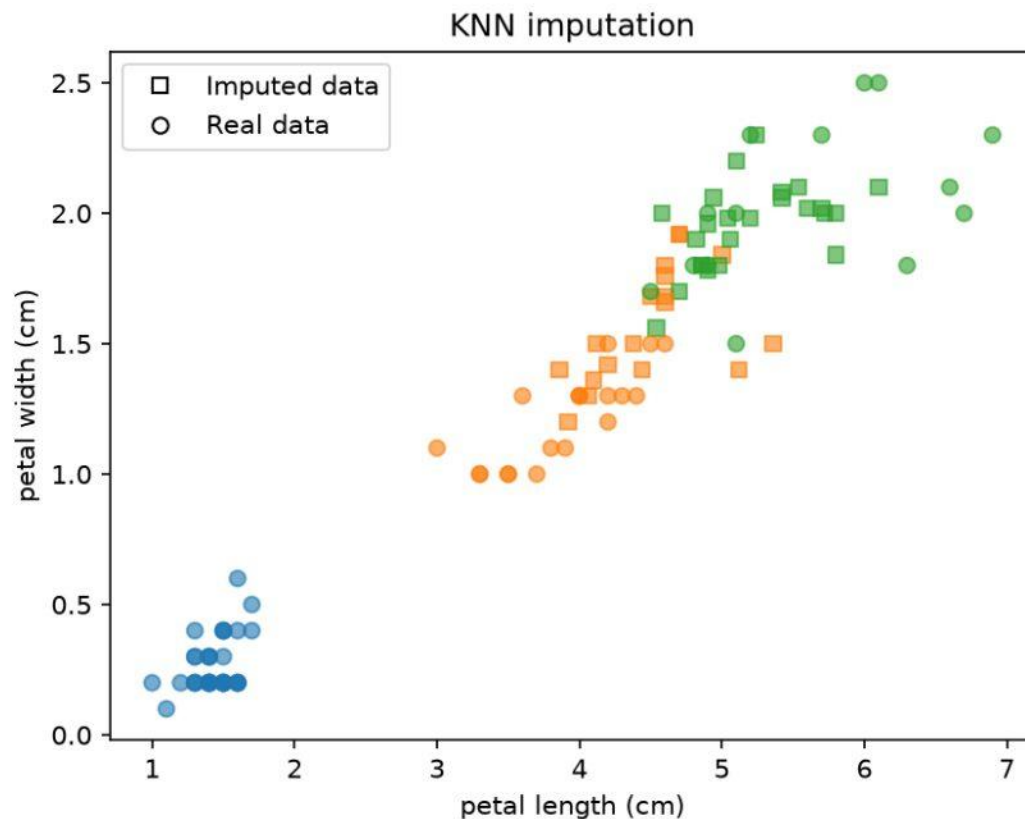
median_ind = make_pipeline(SimpleImputer(strategy='median', add_indicator=True),
                           StandardScaler(), LogisticRegression())
cross_val_score(median_ind, X_train, y_train, cv=10).mean()              # 0.883

knn = make_pipeline(KNNImputer(), StandardScaler(), LogisticRegression())
cross_val_score(knn, X_train, y_train, cv=10).mean()                    # 0.920
```

KNN Imputation

- Find k nearest neighbors that have non-missing values.
- Fill in all missing values using the average of the neighbors.
- Distances ignore missing coordinates (`nan_euclidean_distances`)

KNN Imputation: result



Model-Driven Imputation

- Train a regression model for each feature with missing values
- Iterate: start from mean imputation, re-predict, repeat
- `IterativeImputer` (still marked experimental in scikit-learn)

```
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
rf_imp = IterativeImputer(estimator=RandomForestRegressor(n_estimators=20),
                          max_iter=5)
rf_pipe = make_pipeline(rf_imp, StandardScaler(), LogisticRegression())
cross_val_score(rf_pipe, X_train, y_train, cv=10).mean() # 0.938
```

Data Leakage

Leakage: a reproducibility problem

Kapoor & Narayanan (2023), "Leakage and the reproducibility crisis in machine-learning-based science", *Patterns* 4(9), 100804.

- Survey of ML-based science: leakage affected 294 papers across 17 fields
- Leakage = the evaluation uses information that would not be available when the model is used
- Their taxonomy includes:
 - no clean train/test separation (preprocessing or feature selection on all data, duplicates)
 - features that are proxies of the target, or not available at prediction time
 - test data not drawn from the target distribution (e.g. temporal leakage)

Leak 1: preprocessing fitted on the full data

Feature selection on noise (earlier today):

```
leaky:      CV R^2 =  0.91
pipeline:   CV R^2 = -0.25
held-out:   R^2    = -0.10
```

Target encoding of a random 2,000-level feature:

```
te = TargetEncoder(target_type="contin
enc_leaky = te.fit(X, y).transform(X)
enc_cross = te.fit_transform(X, y)
```

```
fit().transform()  train R^2 = 0.456, test R^2
fit_transform()    train R^2 = 0.002, test R^2
```

Scalers, imputers, encoders, feature selection, PCA, SMOTE: all are fit on training folds only.

Leak 2: target leakage

- A feature that is derived from the target, or only known after it
- House data: adding `price_per_sqft = price / sqft_living`

honest features: $R^2 = 0.693$

with leak: $R^2 = 0.889$

- Real examples: "treatment given" when predicting diagnosis, "account closed date" when predicting churn, hospital-specific scanner tags in medical images
- Check: *at the moment of prediction, would I know this value?*

Leak 3: duplicates across splits

3,000 houses, each copied three times (as after naive oversampling or merging sources):

```
rf = RandomForestRegressor(n_estimators=100, random_state=0)
cross_val_score(rf, X_dup, y_dup, cv=KFold(5, shuffle=True))
cross_val_score(rf, X_dup, y_dup, cv=GroupKFold(5), groups=house_id)
```

```
original,    KFold:      0.831
duplicated,  KFold:      0.984
duplicated,  GroupKFold: 0.818
```

- Also: repeated measurements of the same patient, user, or house (197 houses sold twice here)
- Fix: deduplicate, and split by group (GroupKFold, Week 2)

Leak 4: temporal leakage

- Shuffled splits train on the future and test on the past
- House sales, sorted by date, gradient boosting:

shuffled KFold: $R^2 = 0.886$

TimeSeriesSplit: $R^2 = 0.873$

- Small here (one year, stable market); large under trends, seasonality, or drift
- Also: features computed with future data (rolling means over the whole series)

The fix: the pipeline goes inside cross-validation

```
model = make_pipeline(  
    make_column_transformer(  
        (TargetEncoder(target_type="continuous"), ["zipcode"]),  
        (StandardScaler(), make_column_selector(dtype_include="number"))),  
    Ridge()  
  
cv = TimeSeriesSplit(5)          # or GroupKFold / StratifiedKFold: match deployment  
scores = cross_val_score(model, X_sorted_by_date, y, cv=cv)
```

Checklist:

1. Every step that learns from data is inside the pipeline
2. The splitter matches how the model will be used (time, groups)
3. Deduplicate before splitting; every feature is available at prediction time
4. The test set is touched once, at the end

Tracking the difference: MLflow

```
import mlflow
mlflow.set_experiment("week03-preprocessing")

for run_name, cv_r2, test_r2 in [("feature-selection / leaky", leaky_cv, leaky_test),
                                ("feature-selection / pipeline", correct_cv, correct_test)]:
    with mlflow.start_run(run_name=run_name):
        mlflow.log_params({"percentile": 5, "n_features": 10000, "seed": 0})
        mlflow.log_metrics({"cv_r2": cv_r2, "test_r2": test_r2,
                             "optimism": cv_r2 - test_r2})
        mlflow.set_tag("leakage", "yes" if "leaky" in run_name else "no")
```

- `uv run mlflow ui` → compare runs side by side
- Logging CV and held-out scores together makes optimism visible

Try it 2: Ames housing mini-competition (15 min)

- 1,460 houses, 79 features (43 text), many missing values; target: log sale price
- Build **one pipeline**: `ColumnTransformer`, `make_column_selector`, `SimpleImputer`, `OneHotEncoder(handle_unknown="infrequent_if_exist", min_frequency=...)`, optionally `TargetEncoder`
- Compare variants with 5-fold CV on the training part only
- Evaluate your final choice on the held-out set **once**; post RMSE (log price)
- Baseline, numeric features only: CV RMSE = 0.157

Try it 3: find the leakage (15 min)

A colleague reports an impressive CV R^2 on the house data. In `lab.ipynb`, section "Try it 3":

1. Find every leak in their code and name its kind (from the four above)
2. Write a corrected evaluation: one pipeline, `TargetEncoder` inside it, a time-aware splitter
3. Log both runs to MLflow with a `leakage` tag and compare them

Summary

- Scale features for distance-based and penalized models; trees do not care
- Encode categories with `OneHotEncoder` (infrequent / unknown handling) or `TargetEncoder`
- Check distributions; transform skewed features and targets, then verify with CV
- Impute missing values inside the pipeline; missingness indicators often help
- Put every learned preprocessing step in a `Pipeline` and cross-validate the pipeline
- Leakage: preprocessing on all data, target proxies, duplicates, the future

Next week: linear models, regression, regularization, logistic regression, SVMs

Questions?

Backup

Coming up with features is difficult, time-consuming, requires expert knowledge. "Applied machine learning" is basically feature engineering.

Andrew Ng

Sparse Data

- Many features, most entries zero (counts, clicks, one-hot columns): store only non-zeros.
- Centering subtracts a value from every entry: the matrix becomes dense and may not fit in memory.
- Scale without centering: `MaxAbsScaler` or `StandardScaler(with_mean=False)`.

Going wildest with Pipelines

```
from sklearn.tree import DecisionTreeRegressor
pipe = Pipeline([('scaler', StandardScaler()),
                  ('regressor', Ridge())])

param_grid = [{ 'regressor': [DecisionTreeRegressor()],
                  'regressor__max_depth': [2, 3, 4],
                  'scaler': ['passthrough']},
               { 'regressor': [Ridge()],
                  'regressor__alpha': [0.1, 1],
                  'scaler': [StandardScaler(), MinMaxScaler(),
                              'passthrough']}
               ]

grid = GridSearchCV(pipe, param_grid)
grid.fit(X_train, y_train)
grid.score(X_test, y_test)
```

One-Hot (Dummy) Encoding

	boro	salary	vegan
0	Manhattan	103	No
1	Queens	89	No
2	Manhattan	142	No
3	Brooklyn	54	Yes
4	Brooklyn	63	Yes
5	Bronx	219	No

```
pd.get_dummies(df, columns=['boro'], dtype=int)
```

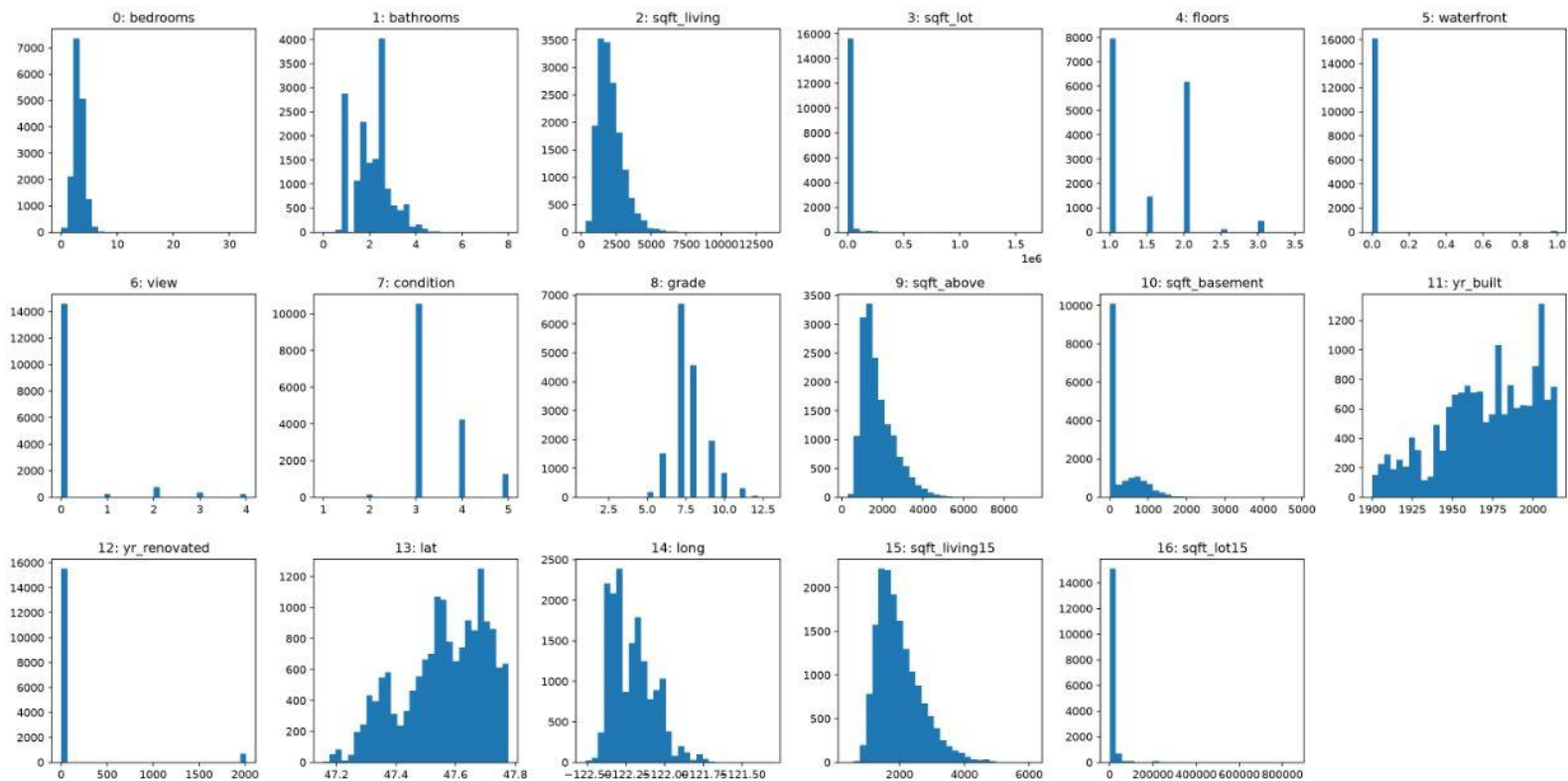
	salary	vegan	boro_Bronx	boro_Brooklyn	boro_Manhattan	boro_Queens
0	103	No	0	0	1	0
1	89	No	0	0	0	1
2	142	No	0	0	1	0
3	54	Yes	0	1	0	0
4	63	Yes	0	1	0	0
5	219	No	1	0	0	0

One-Hot (Dummy) Encoding

	boro	salary	vegan
0	2	103	No
1	3	89	No
2	2	142	No
3	1	54	Yes
4	1	63	Yes
5	0	219	No

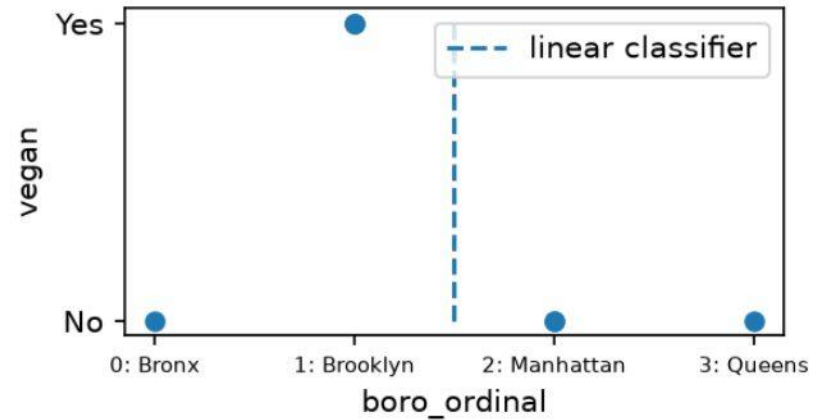
```
pd.get_dummies(df_ordinal, columns=['boro'], dtype=int)
```

	salary	vegan	boro_0	boro_1	boro_2	boro_3
0	103	No	0	0	1	0
1	89	No	0	0	0	1
2	142	No	0	0	1	0
3	54	Yes	0	1	0	0
4	63	Yes	0	1	0	0
5	219	No	1	0	0	0



Ordinal encoding

	boro_ordinal	salary	vegan
0	2	103	No
1	3	89	No
2	2	142	No
3	1	54	Yes
4	1	63	Yes
5	0	219	No



```
df = pd.DataFrame({
    'boro': ['Manhattan', 'Queens', 'Manhattan',
            'Brooklyn', 'Brooklyn', 'Bronx'],
    'salary': [103, 89, 142, 54, 63, 219],
    'vegan': ['No', 'No', 'No', 'Yes', 'Yes', 'No']})
df_dummies = pd.get_dummies(df, columns=['boro'], dtype=int)
```

	salary	vegan	boro_Bronx	boro_Brooklyn	boro_Manhattan	boro_Queens
0	103	No	0	0	1	0
1	89	No	0	0	0	1
2	142	No	0	0	1	0
3	54	Yes	0	1	0	0
4	63	Yes	0	1	0	0
5	219	No	1	0	0	0

```
df = pd.DataFrame({
    'boro': ['Brooklyn', 'Manhattan', 'Brooklyn',
            'Queens', 'Brooklyn', 'Staten Island'],
    'salary': [61, 146, 142, 212, 98, 47],
    'vegan': ['Yes', 'No', 'Yes', 'No', 'Yes', 'No']})
df_dummies = pd.get_dummies(df, columns=['boro'], dtype=int)
```

	salary	vegan	boro_Brooklyn	boro_Manhattan	boro_Queens	boro_Staten Island
0	61	Yes	1	0	0	0
1	146	No	0	1	0	0
2	142	Yes	1	0	0	0
3	212	No	0	0	1	0
4	98	Yes	1	0	0	0
5	47	No	0	0	0	1

Pandas Categorical Columns

```
df['boro'] = pd.Categorical(df.boro,  
                           categories=['Manhattan', 'Queens', 'Brooklyn',  
                                     'Bronx', 'Staten Island'])  
pd.get_dummies(df, columns=['boro'], dtype=int)
```

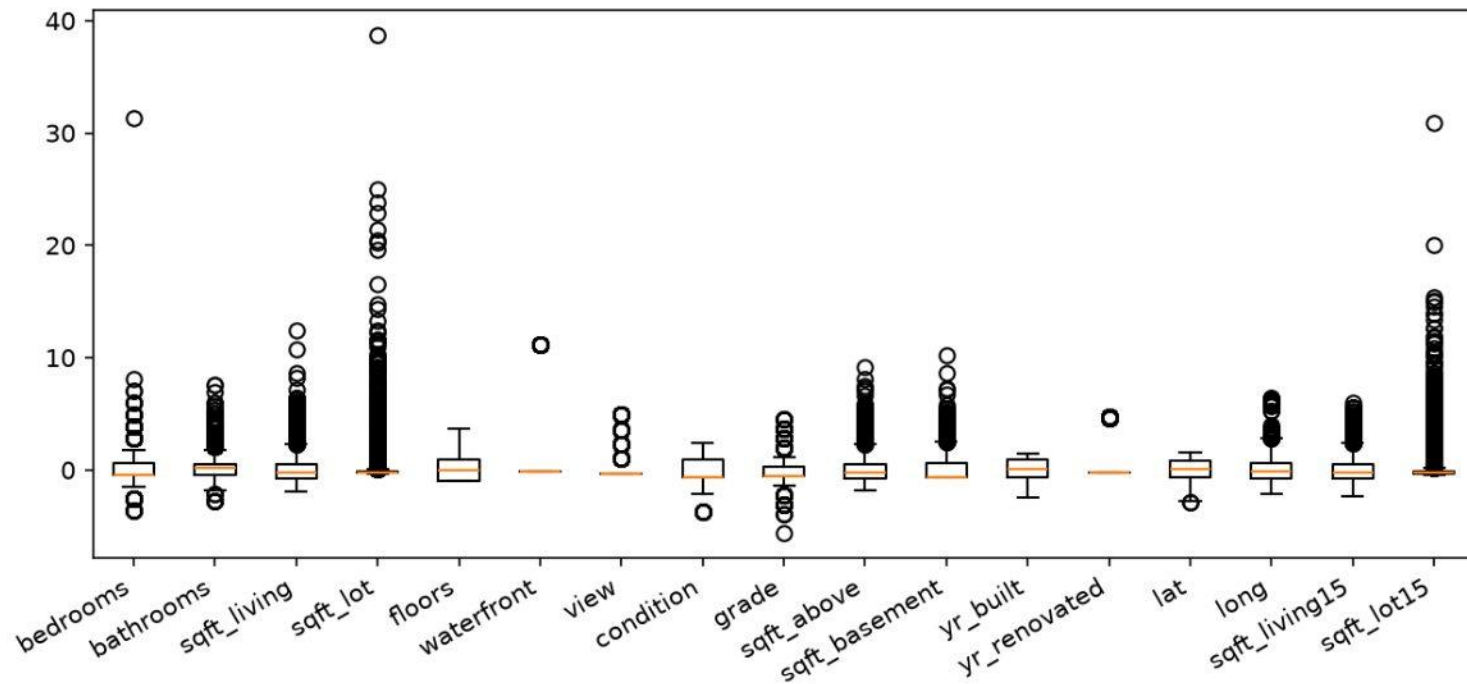
	salary	vegan	boro_Manhattan	boro_Queens	boro_Brooklyn	boro_Bronx	boro_Staten Island
0	103	No	1	0	0	0	0
1	89	No	0	1	0	0	0
2	142	No	1	0	0	0	0
3	54	Yes	0	0	1	0	0
4	63	Yes	0	0	1	0	0
5	219	No	0	0	0	1	0

Dummy variables and collinearity

- One-hot is redundant (last column = $1 - \text{sum of the others}$)
- Unpenalized linear models: exact collinearity, no unique solution
- `OneHotEncoder(drop="first")` or `drop="if_binary"` removes one column
- With regularization, keeping all columns is usually fine and more interpretable
- Which column you drop changes penalized models

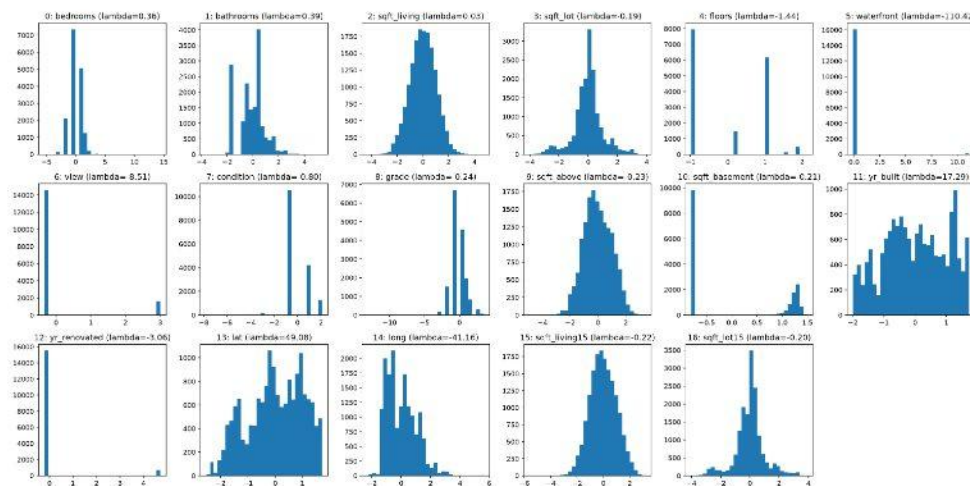
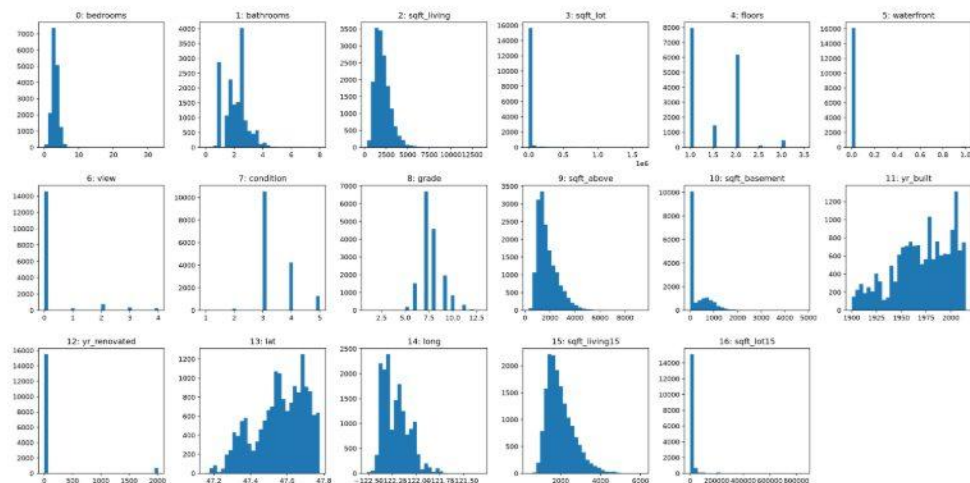
Models Supporting Discrete Features

- In principle: tree-based models, naive Bayes
- scikit-learn:
 - `HistGradientBoostingClassifier/Regressor(categorical_features="from_dt`
 - `DecisionTree*` and `RandomForest*`: need encoding (ordinal is fine for trees)
 - `CategoricalNB`
- LightGBM, CatBoost, XGBoost: native categorical support



Before

After (Yeo-Johnson, λ in titles)



Comparison of Imputation Methods

